

ORF307_HW4_soln

September 25, 2025

ORF307 Homework 4

- Please export your code with output as pdf.
- If there is any additional answers, please combine them as **ONE** pdf file before submitting to the Gradescope.

```
[1]: #from google.colab import drive
#drive.mount('/content/drive')
```

Q1 Fitting a line with different norms

Download the datafile “HW4Q1.csv” for this questions from Canvas/Assignments/HW4/ and use the following code to read the data (you may need to change the path to the one you saved the datafile):

```
[2]: import pandas as pd
import numpy as np
# df = pd.read_csv('/content/drive/MyDrive/HW4Q1.csv')
df = pd.read_csv('HW4Q1.csv')

y = df['y'].values
t = df['t'].values
```

This will generate two vectors $y, t \in \mathbf{R}^{42}$. We are interested in fitting a linear function

$$f(t) = \alpha + \beta t$$

through the points (t_i, y_i) , i.e., we want to select α and β such that $f(t_i) \approx y_i, i = 1, \dots, 42$. We can calculate α and β by optimizing the following three criteria.

- (a) *Least-squares*: select α and β by minimizing

$$\sum_{i=1}^{42} (y_i - \alpha - \beta t_i)^2.$$

- (b) ℓ_1 -norm approximation: select α and β by minimizing

$$\sum_{i=1}^{42} |y_i - \alpha - \beta t_i|.$$

(c) ℓ_∞ -norm approximation: select α and β by minimizing

$$\max_{i=1,\dots,42} |y_i - \alpha - \beta t_i|.$$

Find the optimal values of α and β for each of the three optimization criteria. This yields three linear functions $f_{ls}(t)$, $f_{\ell_1}(t)$, f_{ℓ_∞} . Plot the 42 data points, and the three functions f . What do you observe?

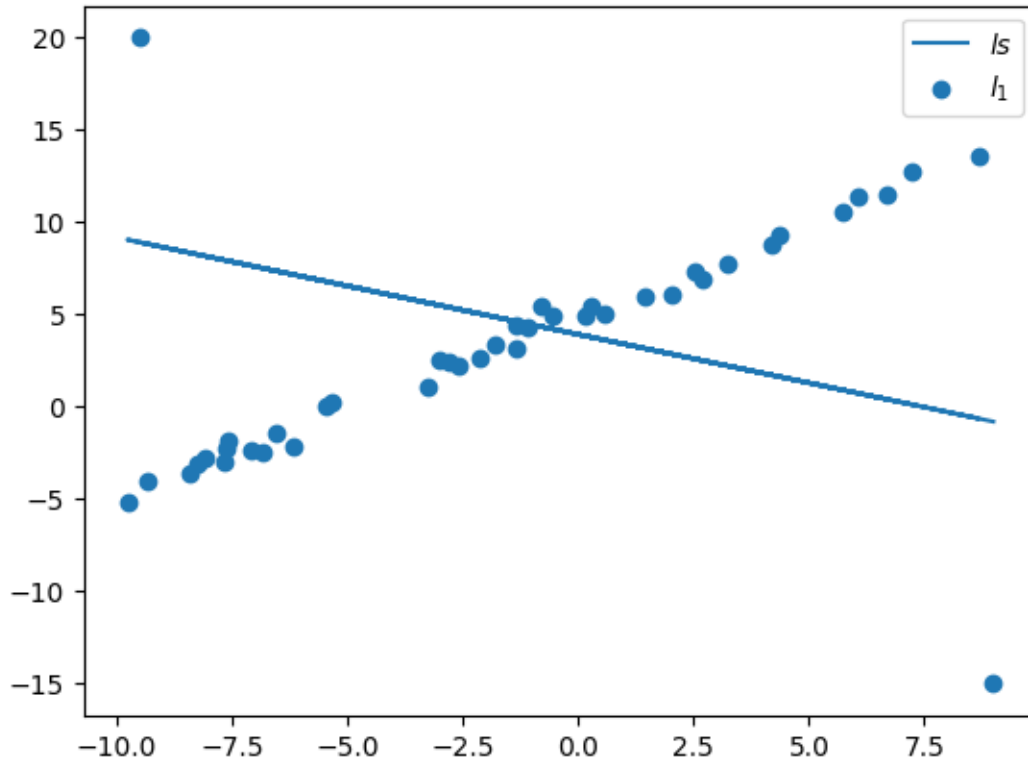
```
[3]: import cvxpy as cp
from matplotlib import pyplot as plt

coeffs = cp.Variable((2,1))
objectives = []
listCoeffVal = []
# (a)
objectives.append(cp.Minimize(cp.sum_squares(y - coeffs[0] - coeffs[1]*t)))
# (b)
objectives.append(cp.Minimize(cp.sum(cp.abs(y - coeffs[0] - coeffs[1]*t))))
# (c)
objectives.append(cp.Minimize(cp.max(cp.abs(y - coeffs[0] - coeffs[1]*t))))

for obj in objectives:
    prob = cp.Problem(obj)
    prob.solve()
    listCoeffVal.append(coeffs.value)

# Plot
plt.plot(t, coeffs.value[0] + coeffs.value[1]*t)
plt.scatter(t, y)
plt.legend(['$ls$', '$l_1$', '$l_\infty$'])
```

```
[3]: <matplotlib.legend.Legend at 0x1c518e5f990>
```



The two outliers are the major sources of discrepancy between the models. The ℓ_∞ -norm model minimizes the maximum deviation thus we see the endpoints of the plot drawn to the outliers. Compared to the ℓ_1 -norm solution, the least-squares (ℓ_2 -norm) solution penalizes larger deviations from the model more, as a result its slope is smaller in order to be “closer” to the outliers.

Q2 Linear programs with absolute values

Consider the problem

$$\begin{aligned} & \text{minimize} && 2x_1 + 3|x_2 - 10| \\ & \text{subject to} && |x_1 + 2| + |x_2| \leq 5 \end{aligned} \tag{1}$$

(a) Reformulate this as a linear program.

We include three new variables y_1, y_2, y_3 to get rid of the three absolute values. For example, $y_1 \geq x_2 - 10$ and $y_1 \geq -x_2 + 10$

$$\begin{aligned}
&\text{minimize} && 2x_1 + 3y_1 \\
&\text{subject to} && y_1 \geq x_2 - 10 \\
& && y_1 \geq -x_2 + 10 \\
& && y_2 \geq x_1 + 2 \\
& && y_2 \geq -x_1 - 2 \\
& && y_3 \geq x_2 \\
& && y_3 \geq -x_2 \\
& && y_2 + y_3 \leq 5
\end{aligned}$$

(b) Reformulate this as a linear program in standard form.

To write this LP in standard form, we need to add slack variables $(s_1, s_2, \dots, s_7)$ to make the inequality constraints equality constraints. Then we need to break x_1 and x_2 into positive and negative parts (e.g. $x_1 = x_1^+ - x_1^-$, $x_1^+ \geq 0, x_1^- \geq 0$).

$$\begin{aligned}
&\text{minimize} && 2x_1^+ - 2x_1^- + 3y_1 \\
&\text{subject to} && -y_1 + x_2^+ - x_2^- + s_1 = 10 \\
& && y_1 + x_2^+ - x_2^- - s_2 = 10 \\
& && y_2 - x_1^+ + x_1^- - s_3 = 2 \\
& && -y_2 - x_1^+ + x_1^- + s_4 = 2 \\
& && -y_3 + x_2^+ - x_2^- + s_5 = 0 \\
& && -y_3 - x_2^+ + x_2^- + s_6 = 0 \\
& && y_2 + y_3 + s_7 = 5 \\
& && x^+ \geq 0, x^- \geq 0, y \geq 0, s \geq 0
\end{aligned}$$

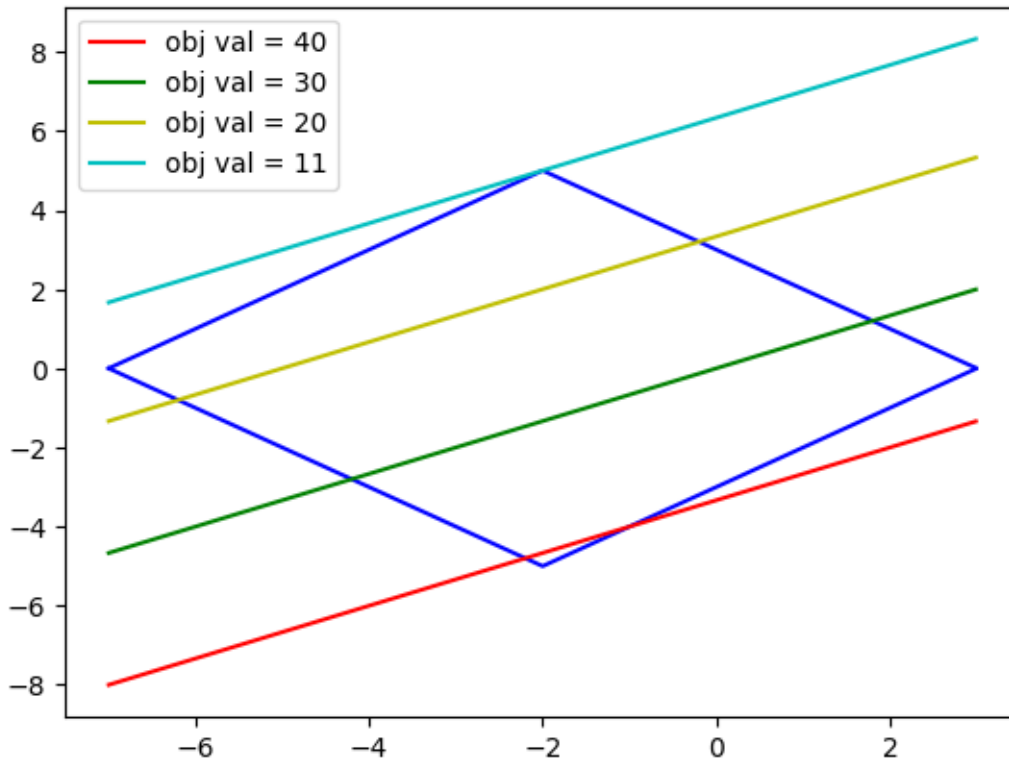
(c) Solve problem (1) graphically.

```
[4]: # plot the constraints
import matplotlib.pyplot as plt
# 7 and 3 chosen because they fully showcase the feasible region
x1 = np.arange(-7, 3, .01)
x2_1st_constraint = 5 - np.abs(x1+2)
x2_2nd_constraint = -5 + np.abs(x1+2)

plt.plot(x1, x2_1st_constraint, 'b')
plt.plot(x1, x2_2nd_constraint, 'b')

# now plot the objective function -- but first notice that x_2 <= 10 for all
# points in the
# feasible region, so we only need to consider x_2 <= 10 and we can get rid of
# the absolute value
# obj = 2*x1 - 3*(x_2 - 10)
# pick several objective values and plot the contour
plt.plot(x1, -(40 - 2*x1)/3+10, 'r', label='obj val = 40')
plt.plot(x1, -(30 - 2*x1)/3+10, 'g', label='obj val = 30')
plt.plot(x1, -(20 - 2*x1)/3+10, 'y', label='obj val = 20')
plt.plot(x1, -(11 - 2*x1)/3+10, 'c', label='obj val = 11')
plt.legend()
```

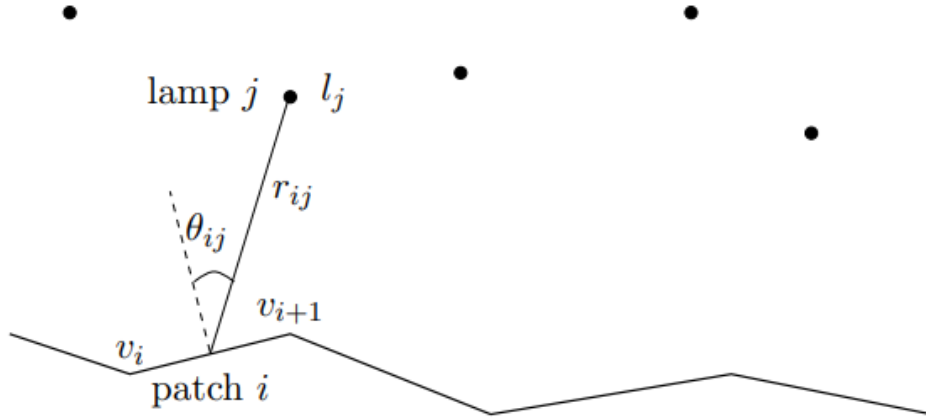
[4]: <matplotlib.legend.Legend at 0x1c51bd87450>



By trying different objective function values (40, 30, 20, 11), we see that 11 is the optimal solution. Any value lower than 11 would violate feasibility.

Q3 Optimal Illumination

We consider an illumination system of m lamps, at positions $l_1, \dots, l_m \in \mathbf{R}^2$, illuminating n flat patches.



The patches are line segments; the i th patch is given by $[v_i, v_{i+1}]$ where $v_1, \dots, v_{n+1} \in \mathbf{R}^2$. The variables in the problem are the lamp powers $p_1, \dots, p_m$, which can vary between 0 and 1. The illumination at (the midpoint of) patch i is denoted I_i . We will use a simple model for the illumination:

$$I_i = \sum_{j=1}^m a_{ij} p_j, \quad a_{ij} = r_{ij}^{-2} \max\{\cos \theta_{ij}, 0\},$$

where r_{ij} denotes the distance between lamp j and the midpoint of patch i , and θ_{ij} denotes the angle between the upward normal of patch i and the vector from the midpoint of patch i to lamp j , as shown in the figure. This model takes into account “self-shading” (*i.e.*, the fact that a patch is illuminated only by lamps in the halfspace it faces) but not shading of one patch caused by another. Of course we could use a more complex illumination model, including shading and even reflections. This just changes the matrix relating the lamp powers to the patch illumination levels.

The problem is to determine lamp powers that make the illumination levels close to a given desired illumination level I_{des} , subject to the power limits $0 \leq p_i \leq 1$.

(a) Suppose we use the maximum deviation

$$\phi(p) = \max_{k=1, \dots, n} |I_k - I_{\text{des}}|$$

as a measure for the deviation from the desired illumination level. Formulate the illumination problem using this criterion as a linear optimization problem.

We introduce the decision variable $s \in \mathbf{R}$, then write:

$$\begin{aligned} & \text{minimize} && s \\ & \text{subject to} && s \geq \sum_{j=1}^m a_{kj} p_j - I_{\text{des}} \geq -s, \quad \forall k = 1, \dots, n \\ & && 1 \geq p_j \geq 0, \quad \forall j = 1, \dots, m. \end{aligned}$$

(b) There are several suboptimal approaches based on weighted least-squares. You are given A (where the entries of A are a_{ij} in the model for illumination) and I_{des} . Download the datafile

“HW4Q3.csv” from Canvas/Assignments/HW4 and read the data using the following code (you may need to change the path to the one you saved the datafile):

```
[5]: # A_pd = pd.read_csv('/content/drive/MyDrive/HW4Q3.csv', header=None)
A_pd = pd.read_csv('HW4Q3.csv', header=None)
A = A_pd.to_numpy()
I_des = 2
```

We consider two examples.

1. *Saturated least-squares.* We can solve the least-squares problem

$$\text{minimize} \quad \sum_{k=1}^n (I_k - I_{\text{des}})^2$$

ignoring the constraints. If the solution is not feasible, we saturate it, *i.e.*, set $p_j = 0$ if $p_j \leq 0$ and $p_j = 1$ if $p_j \geq 1$. Compute a feasible p using this first method, and calculate $\phi(p)$.

2. *Weighted least-squares.* We consider another least-squares problem:

$$\text{minimize} \quad \sum_{k=1}^n (I_k - I_{\text{des}})^2 + \mu \sum_{i=1}^m (p_i - 0.5)^2,$$

where $\mu \geq 0$ is used to attach a cost to a deviation of the powers from the value 0.5, which lies in the middle of the power limits. For large enough μ , the solution of this problem will satisfy $0 \leq p_i \leq 1$, *i.e.*, be feasible for the original problem. Again, using the same A and I_{des} , find the smallest μ such that p becomes feasible, and evaluate $\phi(p)$.

```
[6]: def evalPhi(p, A, I_des):
    I_k = A@p
    return max(abs(I_k - I_des))

# b1
m = A.shape[1]
n = A.shape[0]
p = cp.Variable((m,1))
obj_b1 = cp.Minimize(cp.sum_squares(A@p - I_des))
prob_b1 = cp.Problem(obj_b1)
prob_b1.solve()
print('There are some entries in p greater than 1 and some less than 0: \n', p.
      ↪value)
p.value[p.value>=1] = 1
p.value[p.value<=0] = 0
print('Value of p after processing: \n', p.value)
print('Phi(p): ', evalPhi(p.value, A, I_des))
print('\n')

# b2
mu=7.3
obj_b2 = cp.Minimize(cp.sum_squares(A@p-I_des) + mu*cp.sum_squares(p-0.5))
```

```

prob_b2 = cp.Problem(obj_b2)
prob_b2.solve()
print('Value of p: \n', p.value)
print('Phi(p): ', evalPhi(p.value, A, I_des))

```

There are some entries in p greater than 1 and some less than 0:

```

[[ 5.26966575]
 [-2.45573954]
 [ 0.52322558]
 [-8.12880821]
 [11.67060976]
 [-5.66888204]
 [-1.42315408]]

```

Value of p after processing:

```

[[1.      ]
 [0.      ]
 [0.52322558]
 [0.      ]
 [1.      ]
 [0.      ]
 [0.      ]]

```

Phi(p): [3.24695557]

Value of p:

```

[[ 0.60361662]
 [ 0.5003586 ]
 [-0.02211918]
 [ 0.34445844]
 [ 0.38777075]
 [ 0.46907505]
 [ 0.45914574]]

```

Phi(p): [1.30430795]

[7]: I_des

[7]: 2

[8]: A

[8]: array([[0.69483074, 0.41069986, 0.38410511, 0.15443517, 0.00916266,
0.04803079, 0.],
[0.97217829, 0.57819842, 0.75868342, 0.23550446, 0.01339488,
0.06454695, 0.],
[1.50500273, 1.22257337, 4.83611427, 1.403457 , 0.94613895,
0.57121288, 0.63840044],
[0.60716988, 0.72925165, 3.79641008, 1.3863946 , 1.21548896,
0.65718704, 0.87482388],


```
[1.08191029, 0.8231571 , 3.39358194, 0.78363859, 0.35580535,
 0.22030357, 0.16772274],
[1.30344901, 1.08100541, 6.41560884, 1.27708103, 0.58669588,
 0.31324246, 0.25636096],
[0.19613316, 0.49166131, 1.48612843, 1.61614496, 1.85952307,
 0.87030902, 1.33958934],
[0.          , 0.05398322, 0.          , 0.49332918, 0.82940328,
 0.46359117, 0.74444901],
[0.57121288, 0.64092806, 1.70254085, 1.16936954, 1.23286934,
 0.68288952, 0.95464769],
[0.64926338, 0.61280701, 1.98388105, 0.95552107, 0.75256232,
 0.31659509, 0.39751472],
[0.          , 0.10076251, 0.          , 0.42416225, 0.86887711,
 0.57294845, 0.92551863]])
```

[]:

(c) Using the same data as in part (b), solve the LP you derived in part (a). Compare the solution with the solutions you obtained using the (weighted) least-squares methods of part (b).

```
[9]: p,s = cp.Variable((m,1)), cp.Variable(1)
constraints = []
for k in range(n):
    constraints += [s >= A[k,:]*p - I_des, A[k,:]*p - I_des >= -s]
constraints += [1 >= p, p >= 0]
obj = cp.Minimize(s)
prob = cp.Problem(obj, constraints)
prob.solve()
print('Value of p: \n', p.value)
print('Phi(p): ', prob.value)
```

Value of p:

```
[[9.99999990e-01]
 [5.70808574e-01]
 [2.65742121e-10]
 [7.83801665e-09]
 [1.95973426e-08]
 [3.70580701e-01]
 [9.99999945e-01]]
```

Phi(p): 1.0529389803867861

[]:

The solution from the LP is the best, followed by that computed from the weighted-least squares. The saturated-least squares formulation yields the worst solution.